“Linux Process Creation, Execution and Termination System”

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

ABHAY TRIPATHI	**2520030353**
K. PRANAV	**2520030477**
JASHWANTH	**2520030236**

Team 08 | Section 4 | Academic Year 2026-2027

Under the Esteemed Guidance of

Ms. Soumya

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **“Linux Process Creation, Execution and Termination System”** is a record of the work of **Abhay Tripathi - 2520030353, K. Pranav - 2520030477, and Jashwanth - 2520030236**, submitted in partial fulfillment of the requirements for the award of B.Tech in Computer Science and Engineering at K L (Deemed to be) University.

The report documents the Team 08 project for Operating Systems and System Programming (OSSP), course code 25CS2104E. It demonstrates the Linux process lifecycle through ProcessPay, an educational payment simulation that does not process real payments or connect to banking services.

ABHAY TRIPATHI	**2520030353**
K. PRANAV	**2520030477**
JASHWANTH	**2520030236**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **“Linux Process Creation, Execution and Termination System”** is submitted by **Abhay Tripathi - 2520030353, K. Pranav - 2520030477, and Jashwanth - 2520030236**, Team 08, Section 4, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science Engineering, K L (Deemed to be) University, for the course Operating Systems and System Programming (25CS2104E), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to the Department of Computer Science and Engineering at K L (Deemed to be) University for providing the academic environment and resources needed to complete this Operating Systems and System Programming project.

We thank our project guide, course coordinator, and faculty members for their instruction and feedback on operating-system concepts, especially Linux process management, inter-process communication, synchronization, and process termination. We also acknowledge the support of our classmates and families during the development and demonstration of the project.

The project was built as a team learning exercise. We are grateful for the open technical documentation maintained by the GNU and Linux communities, which supports understanding and responsible use of the system interfaces discussed in this report.

ABHAY TRIPATHI **2520030353**
K. PRANAV **2520030477**
JASHWANTH **2520030236**

Abstract

This project, Linux Process Creation, Execution and Termination System, demonstrates real Linux process lifecycle behaviour using a simulated payment workflow named ProcessPay. The project is implemented in C and runs on Linux or Ubuntu under WSL. It gives a practical, observable demonstration of process creation with fork(), program-image replacement with execl(), parent–child communication using pipes, blocking input, signal-based termination, exit-status inspection, and child reaping.

When the user starts a payment demonstration, the parent application creates a child process. The child replaces its program image with a worker while retaining its PID. The worker blocks while waiting for an OTP delivered through a pipe. A correct OTP produces normal exit status 0; an incorrect OTP produces normal exit status 1; and a decline sends SIGTERM to the child. The parent uses waitid() with WNOWAIT to observe the terminated child before using waitpid() to collect its status and release the zombie entry.

The program reads live process information from Linux /proc, including PID, PPID, process state, UID values, thread count, resident memory where available, process group, session, and nice value. The repository validation record reports passing integration checks for success, incorrect OTP, decline, PID/PPID relationships, PID continuity across exec, sleeping and zombie states, reaping, interactive payment modes, input EOF cleanup, and invalid amount handling. The application is explicitly educational: it does not move money, contact banks, or use a production OTP service.

Keywords: Linux, operating systems, process lifecycle, fork, exec, pipes, IPC, SIGTERM, waitid, waitpid, zombie process, /proc, C programming.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	1
		1.4 Objectives of the Project	2
		1.5 Importance of the Application	2
		1.6 Target Users / Audience	2
2	Os Concepts And Background Study		3
		2.1 Operating System Concepts Related to the Project	3
		2.2 Existing Methods / Algorithms	4
		2.3 Proposed Approach	4
		2.4 Advantages of the Proposed Approach	4
3	System Requirements		5
		3.1 Hardware Requirements	5
		3.2 Software Requirements	5
		3.3 Operating System Environment - Ubuntu / WSL	5
		3.4 Development Tools and Compilers	5
4	Technology And Tools Used		6
		4.1 C / C++ Programming	6
		4.2 GCC Compiler	6
		4.3 Linux System Calls	6
		4.4 POSIX Threads / Synchronization	6
		4.5 Linux Commands and Utilities	7
		4.6 Other Tools / Libraries Used	7
5	System Architecture And Design		8
		5.1 Overall System Architecture	8
		5.2 System Workflow	8
		5.3 Flowchart / Data Flow	9
		5.4 Module Description	9
		5.5 Input and Output Design	9

Index continued

S. No.	Chapter	Topics / Subtopics	Page No.
6	Implementation		10
		6.1 Module-wise Implementation	10
		6.2 Algorithm / Pseudocode	11
		6.3 Core C / C++ Implementation	11
		6.4 Linux System Calls / APIs Used	12
		6.5 Process / Thread Implementation	12
		6.6 IPC / Synchronization Implementation	12
		6.7 Memory Management Implementation	12
		6.8 Optional User Interface Implementation	12
7	Execution And Demonstration		13
		7.1 Compilation in Ubuntu	13
		7.2 Execution Commands	13
		7.3 Sample Inputs and Outputs	13
		7.4 Screenshots of Execution	14
		7.5 Demonstration of OS Concepts	14
		7.6 Validation and Test Results	14
8	Conclusion		15
		8.1 Summary of the Project	15
		8.2 Objectives Achieved	15
		8.3 Results Achieved	16
		8.4 Skills and Learning Outcomes	16
		8.5 Limitations and Future Enhancements	16
9	References		17
		9.1 Books	17
		9.2 Research Papers	17
		9.3 Linux / GNU Documentation	17
		9.4 Online Resources	17

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

An operating system manages processes that execute programs and share a machine's resources. Linux exposes a set of process-related interfaces that allow a program to create a child process, replace a process image, communicate through file descriptors, observe termination, and collect a child's status. These operations are fundamental to shells, command execution, service supervision, and many server applications.

The project translates these concepts into a familiar scenario: a customer initiates a simulated payment, the parent application creates a worker process, and the worker waits for an OTP. The scenario is not intended to implement a real payment service. It is a teaching interface that makes operating-system behaviour easier to observe.

1.2 Problem Statement

Process APIs are often first encountered as isolated calls in lecture examples. Without observing process identifiers, parent-child relationships, blocking states, exit status, signals, and zombie cleanup in a connected workflow, it can be difficult to understand how these mechanisms interact. The project addresses this learning gap through a reproducible, instrumented demonstration.

1.3 Scope of the Project

- Create a child process from a parent using fork().
- Execute a worker program using execl() while preserving the child PID.
- Exchange readiness and OTP data between parent and child through pipes.
- Observe a child waiting for pipe input and inspect process metadata from /proc.
- Demonstrate normal success exit, normal nonzero exit, and SIGTERM termination.
- Observe the zombie state before reaping and then collect status with waitpid().
- Provide command-line demonstrations, automated integration checks, and a browser-based interface backed by the local Linux program.

Out of scope are real financial transactions, banking integrations, production OTP delivery, persistent transaction records, OTP retries or timeout policies, and guarantees for cleanup after forced termination of the parent or host.

1.4 Objectives of the Project

- Demonstrate the lifecycle from process creation through execution and termination.
- Distinguish process creation (fork) from program-image replacement (exec).
- Explain parent-child communication and synchronization using pipes.
- Show how blocked I/O differs from a parent waiting for child termination.
- Decode normal exit status separately from signal-based termination.
- Demonstrate why waitpid() is required to reap a terminated child.
- Validate the demonstration with repeatable integration tests.

1.5 Importance of the Application

The project combines several core OS concepts in a single workflow. It supports practical learning of process identifiers, scheduling-visible states, file descriptors, IPC, signals, exit codes, and process-table cleanup. Its observable traces make it suitable for classroom demonstrations and viva discussions.

1.6 Target Users / Audience

The intended audience is students studying operating systems and system programming, instructors demonstrating Linux process APIs, and learners who want to connect textbook process-lifecycle concepts with a running Linux program.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process and process identifiers

A process is an executing instance of a program with a kernel-managed execution context and resources. Linux assigns a process identifier (PID). The parent process identifier (PPID) records the relationship to the process's parent. The project prints the parent and child PIDs and reads process metadata through /proc.

Process creation with fork()

fork() creates a child process. On success, it returns 0 in the child and the child's PID in the parent. Both paths continue from the return point, so the program must explicitly branch to separate parent and child responsibilities. The child branch closes unused pipe ends and invokes exec().

Program-image replacement with exec

The exec family replaces the calling process's program image with another program. It does not create a new process. In this implementation, execl("/proc/self/exe", ...) starts the worker mode of the same executable; the child PID remains unchanged. If exec fails, the child reports the error and exits using _exit(127).

Inter-process communication and blocking

Pipes provide byte-stream communication through file descriptors. The readiness pipe lets the parent know that the worker initialized. The OTP pipe carries the user's response. When the OTP pipe contains no data, read() blocks and the child is commonly observed in Linux state S (interruptible sleep).

Signals and termination

The decline path sends SIGTERM to the child with kill(). In the documented sample run, the child terminates from signal 15. A signal requests or causes a process action; it should not be described as identical to a normal return from main().

Exit status, zombie state, and reaping

After a child terminates, the kernel retains status information until the parent collects it. The project uses waitid(P_PID, ..., WEXITED | WNOWAIT) to observe termination without immediately reaping the child, then samples the zombie state. It subsequently calls waitpid() and decodes the status with WIFEXITED/WEXITSTATUS or WIFSIGNALED/WTERMSIG.

2.2 Existing Methods / Algorithms

A basic process-creation demonstration typically calls fork(), prints a PID, and exits. A separate example may show exec() or wait(). Such examples are useful for introducing individual calls, but they may not show IPC, blocking input, multiple termination modes, and the difference between observing a zombie and reaping it in one sequence.

2.3 Proposed Approach

The project uses a parent-worker design embedded in a simulated payment flow. It creates two pipes, forks a child, re-executes the child in worker mode, coordinates readiness, waits for OTP input, handles success/failure/decline paths, observes termination with WNOWAIT, and finally reaps the child. A small web interface can present the trace, while the core process behaviour remains implemented by the C program on Linux.

2.4 Advantages of the Proposed Approach

- Connects multiple OS APIs in one coherent, easy-to-explain scenario.
- Uses real Linux processes and /proc observations rather than simulated PID values.
- Separates input blocking from parent-side child waiting.
- Demonstrates both normal exit and signal-based termination.
- Includes reproducible demo modes and automated integration checks.
- Makes limitations explicit: the payment scenario is educational and not a financial application.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement / Notes
Processor	Any modern x86-64 or compatible processor capable of running Linux/Ubuntu.
Memory	Sufficient memory for a standard Linux desktop or WSL environment; the program itself is lightweight.
Storage	A small amount of free space for source files, executable, test scripts, and optional web assets.
Display / browser	Required only when using the optional browser interface.

The repository does not specify a minimum CPU model, RAM size, or disk capacity. The values above are practical qualitative requirements rather than benchmarked minimum specifications.

3.2 Software Requirements

Software	Role
Linux / Ubuntu	Provides POSIX process APIs, signals, pipes, and /proc.
Ubuntu on WSL	Supported option for Windows users who need a Linux environment.
GCC	Compiles payment.c into the payment executable.
GNU Make	Runs the build and test targets.
Python 3	Runs integration tests and the optional local web server/tests.
Modern web browser	Optional: opens the local website interface.

3.3 Operating System Environment - Ubuntu / WSL

The program depends on Linux-specific behaviour and interfaces, including /proc process metadata and POSIX process management. The repository recommends Linux or Ubuntu under WSL. On Ubuntu, missing build tools can be installed with the package manager command below.

```
sudo apt install build-essential python3
```

3.4 Development Tools and Compilers

The supplied Makefile compiles the C source with GCC using C11 mode, warning flags, and optimization. The recorded build command is:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 payment.c -o payment
```

The project repository also includes Python integration tests, captured sample outputs for the three main outcomes, a local website server, and front-end assets. These tools support implementation, demonstration, and validation.

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The core lifecycle implementation is written in C. C provides direct access to POSIX and Linux APIs, integer exit codes, process identifiers, file descriptors, and signal handling. The program uses standard library routines for input validation, string comparison, output, and error reporting.

4.2 GCC Compiler

GCC compiles payment.c into the payment executable. The Makefile enables C11 language mode and the -Wall, -Wextra, and -Wpedantic diagnostics, with -O2 optimization. Compiler warnings are useful for identifying suspicious code during development.

4.3 Linux System Calls

Interface	Purpose in this project
fork()	Creates the child payment process.
execl()	Replaces the child process image with worker mode.
pipe()	Creates the OTP and readiness communication channels.
read() / write()	Transfers readiness and OTP bytes; read blocks when input is unavailable.
kill() + SIGTERM	Requests termination of the child when payment is declined.
waitid() + WNOWAIT	Waits for child termination while leaving status available for observation.
waitpid()	Collects the child status and reaps the child.
exit(127) _	Terminates the child if exec fails.
/proc/<pid>/status	Supplies process name, state, PID/PPID, UID values, thread count, and VmRSS when available.
getpgid(), getsid(), getpriority()	Read process-group, session, and nice-priority information.

4.4 POSIX Threads / Synchronization

The project does not create POSIX threads. Its synchronization is between separate processes and is achieved with pipes and wait interfaces. The readiness pipe coordinates worker initialization before the parent proceeds to the OTP interaction. waitid() and waitpid() coordinate observation and collection of child termination.

4.5 Linux Commands and Utilities

Command	Purpose
make	Builds the payment executable using the Makefile.
./payment	Runs the interactive command-line demonstration.
./payment --demo success	Runs the correct-OTP outcome without interactive input.
./payment --demo wrong-otp	Runs the incorrect-OTP outcome.
./payment --demo decline	Runs the SIGTERM decline outcome.
make test	Runs the integration test suite.
ps -o pid,ppid,pgid,sid,stat,ni ,rss,comm -p CHILD PID _	Independently inspects a live child process.
cat /proc/CHILD PID/status _	Reads Linux process status information.

4.6 Other Tools / Libraries Used

The optional browser interface uses a local Python server and HTML, CSS, and JavaScript assets. The browser communicates with the local server, which bridges input/output to the C payment application. The core C process lifecycle does not require third-party Python packages or npm dependencies according to the repository documentation.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system consists of a shell-launched parent process, a child worker process, two pipes, Linux process metadata, and a parent-controlled cleanup sequence. An optional web layer provides a more visual interaction path but delegates the real process lifecycle to the Linux C program.

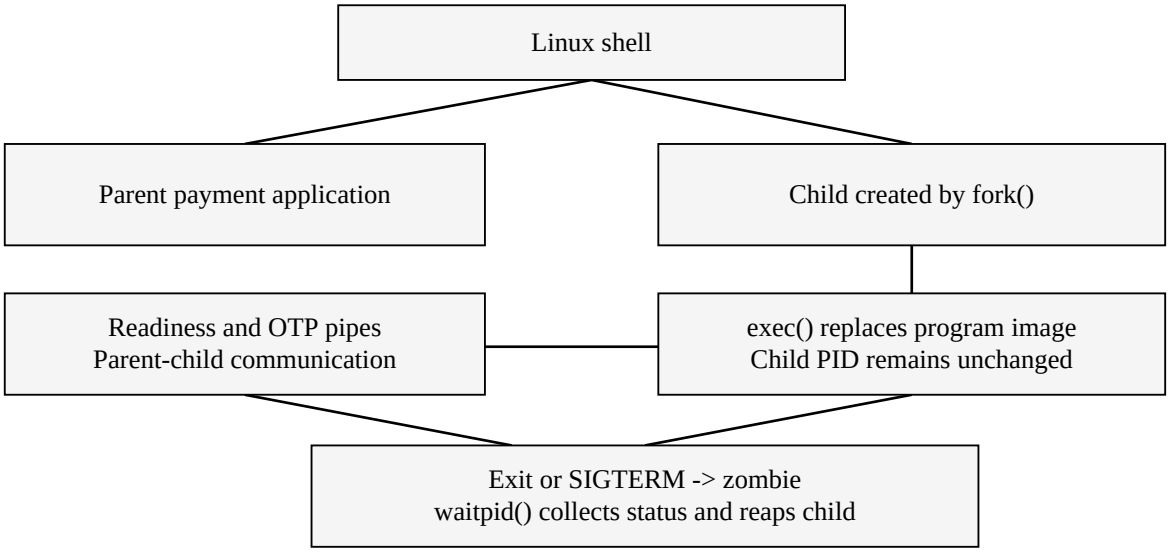


Figure 5.1. Process architecture of the ProcessPay demonstration.

5.2 System Workflow

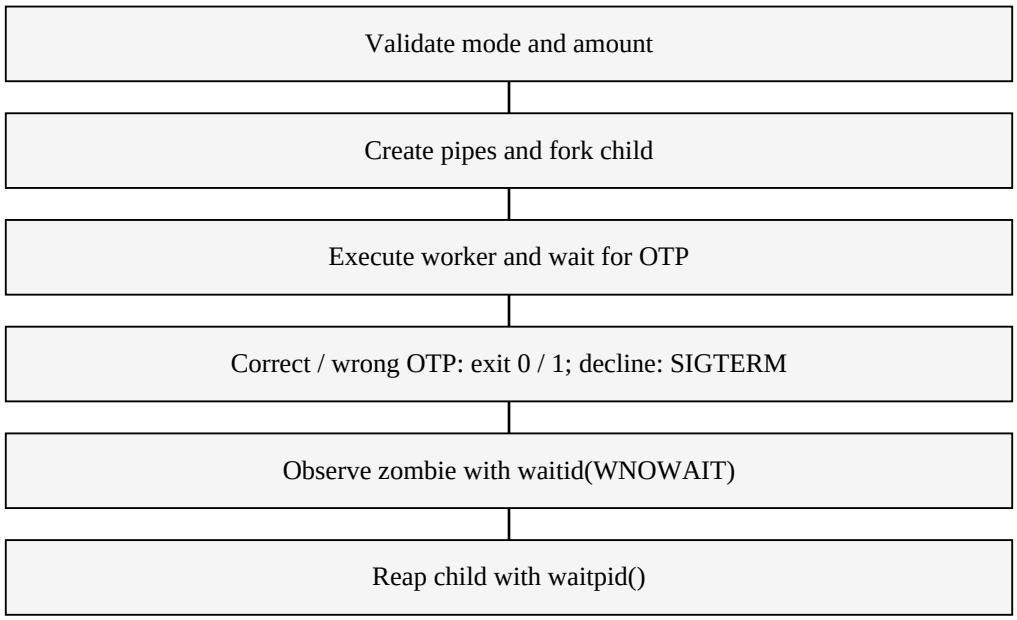


Figure 5.2. High-level lifecycle and cleanup workflow.

5.3 Flowchart / Data Flow

The OTP pipe carries user input from the parent to the worker. The readiness pipe carries a one-byte readiness signal from the worker to the parent. The parent closes unused pipe ends and the worker closes its unused ends so EOF and blocking behaviour remain well-defined. The decline path sends SIGTERM rather than writing an OTP.

5.4 Module Description

Module	Responsibilities
Input and validation	Reads payment mode and amount; accepts integer amounts from 1 to 1,000,000 demo units; requires the user to enter pay before creating a child.
Process creation	Creates OTP/readiness pipes and forks the child.
Worker execution	Re-executes the same executable in worker mode and displays process details.
IPC and synchronization	Uses readiness and OTP pipes to coordinate parent and child.
Process observation	Reads selected fields from /proc/<pid>/status and queries PGID, SID, and nice value.
Termination and status	Handles normal worker exit or SIGTERM, observes zombie status, decodes status, and reaps the child.
Optional website	Provides a browser interface and local server bridge for displaying the live execution trace.

5.5 Input and Output Design

Interactive inputs are payment mode (UPI, Card, or Net banking), a whole-number amount within the permitted demo range, the command pay, and then an OTP or the word decline. A correct OTP is 123456. Any other OTP produces a normal nonzero exit. The payment modes are labels for the same OS demonstration and do not initiate financial transactions.

Input / action	Expected process-level output
Correct OTP: 123456	Worker returns 0; parent reports WIFEXITED=true and WEXITSTATUS=0.
Incorrect OTP	Worker returns 1; parent reports WIFEXITED=true and WEXITSTATUS=1.
decline	Parent sends SIGTERM; sample run reports WIFSIGNALED=true and WTERMSIG=15.
Invalid mode / amount	Input is rejected before a payment child is created.
EOF at OTP prompt	Treated as a decline path to clean up the worker.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

Input validation

The interactive path validates the payment mode and parses the amount with strtol(). It rejects missing or malformed values and values outside 1..1,000,000. The program creates no payment child unless the user confirms the pay action.

Pipe creation and fork

Two pipes are created before fork(): one for OTP input and one for readiness. The child closes the unused write/read ends, prints a pre-exec snapshot, builds string arguments for the pipe file descriptors, and calls execl(). The parent closes its unused ends and waits for the worker readiness byte.

Worker mode

After exec(), the worker prints a process snapshot and waits for OTP bytes using read(). EINTR is retried. Once the line is read, the worker compares the OTP to 123456 and returns 0 for success or 1 for an incorrect OTP. EOF at the worker input returns a separate nonzero code; the parent handles interactive EOF by requesting decline.

Termination and cleanup

For decline, the parent sends SIGTERM. It then uses waitid() with WEXITED | WNOWAIT, inspects the still-waitable child as a zombie, and calls waitpid(). It decodes the final status using the WIFEXITED/WEXITSTATUS and WIFSIGNALED/WTERMSIG macros. Interrupted wait operations are retried.

6.2 Algorithm / Pseudocode

```
START
  Print parent process snapshot
  If interactive:
      Read payment mode and validate
      Read amount and validate range
      Require command "pay"
  Create OTP pipe and readiness pipe
  child_pid = fork()
  IF fork failed: report error and stop
  IF child:
      Close unused pipe ends
      Print pre-exec snapshot
      exec same executable in worker mode
      If exec fails, _exit(127)
  ELSE parent:
      Close unused pipe ends
      Wait for readiness byte
      Observe child waiting for OTP
      IF user declines:
          send SIGTERM to child
      ELSE:
          write OTP line to pipe
      close OTP pipe
      waitid(..., WEXITED | WNOWAIT)
      observe terminated child / zombie state
      status = waitpid(child_pid)
      decode normal exit or terminating signal
      report reaping
END
```

6.3 Core C / C++ Implementation

The following excerpt shows the essential fork/exec structure from the repository. Error checks and pipe cleanup are included in the complete payment.c source.

```
pid_t child = fork();
if (child == -1) fail("fork");
if (child == 0) {
    close(otp_pipe[1]);
    close(ready_pipe[0]);
    /* Prepare descriptor arguments for worker mode. */
    execl("/proc/self/exe", "payment", "--worker",
          input_fd, ready_fd, mode, amount, (char *)NULL);
    perror("exec");
    _exit(127);
}
/* Parent closes unused ends, coordinates readiness,
   sends OTP or SIGTERM, and collects child status. */
```

The worker path returns 0 for the correct OTP and 1 for an incorrect OTP. The parent remains a separate process and prints the child's outcome. The parent demonstration itself returns 0 after completing the lifecycle, even when the simulated payment is declined; its own exit code must not be confused with the worker's status.

6.4 Linux System Calls / APIs Used

The core APIs and their project roles are listed in Section 4.3. A key distinction is that exec() replaces a process image, while fork() creates a child. Similarly, the worker's blocking read() is input waiting, whereas waitid()/waitpid() are parent-side child lifecycle interfaces.

6.5 Process / Thread Implementation

The project uses two processes in the primary workflow: the parent payment application and the child payment worker. The child PID is expected to remain the same across exec(). The project does not create POSIX threads; the Threads field reported by /proc describes the observed process's thread count, which is one in the supplied sample runs.

6.6 IPC / Synchronization Implementation

The readiness pipe prevents the parent from prompting for an OTP before the worker has initialized. The OTP pipe sends the user response. The parent closes the pipe ends it does not use, and the child closes its unused ends. Reads and waits retry when interrupted by EINTR. The parent ignores SIGPIPE before writing the OTP so a worker that has already exited does not terminate the parent unexpectedly.

6.7 Memory Management Implementation

The project does not implement a custom memory allocator or manual memory-management subsystem. The Linux kernel manages each process's address space. The application reports VmRSS from /proc when available. A zombie has terminated and no longer has the worker's active address space; some live-memory fields may therefore be absent from its snapshot.

6.8 Optional User Interface Implementation

The repository includes a browser-based interface, a Python local server, and front-end assets. The browser refreshes execution information and presents the process trace. The server starts or communicates with the C application and bridges its input/output. The UI is optional; the command-line application remains the core implementation.

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

From the project directory, compile the program with Make:

```
make
```

The equivalent recorded compiler command is:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 payment.c -o payment
```

7.2 Execution Commands

```
./payment
./payment --demo success
./payment --demo wrong-otp
./payment --demo decline
make test
```

The interactive mode prompts for UPI, Card, or Net banking, then a whole-number amount, then the pay command. At the OTP prompt, enter 123456 for success, a different OTP for a normal failure, or decline for SIGTERM termination.

7.3 Sample Inputs and Outputs

Scenario	Worker outcome	Status observed by parent
Success - OTP 123456	Normal exit	WIFEXITED=true; WEXITSTATUS=0
Wrong OTP	Normal nonzero exit	WIFEXITED=true; WEXITSTATUS=1
Decline	SIGTERM termination	WIFSIGNALED=true; WTERMSIG=15

Sample output excerpt - successful run

```
PAYMENT SUCCESSFUL: exit(0), normal process exit.
TERMINATED: zombie before parent reaps it
State:  Z (zombie)
REAPED: waitpid() returned PID=507; parent PID=506; raw status=0
WIFEXITED=true | WEXITSTATUS=0
Child reaped: its process-table entry is released; parent remains alive.
```

The PIDs above are reproduced from a captured sample run and are illustrative only; PIDs and memory measurements vary between executions and machines.

Sample output excerpt - decline run

```
DECLINED: kill(509, SIGTERM); signal=15
TERMINATED: zombie before parent reaps it
State:  Z (zombie)
REAPED: waitpid() returned PID=509; parent PID=508; raw status=15
WIFSIGNALED=true | WTERMSIG=15 (Terminated)
```

7.4 Screenshots of Execution

The repository provides captured terminal logs for the success, wrong-OTP, and decline paths, plus a validation record. These logs are used here as textual execution evidence rather than represented as fabricated screenshots. For a submission that specifically requires screenshots, capture the live program in Ubuntu/WSL and insert the original screenshots before final submission.

7.5 Demonstration of OS Concepts

Demonstration step	OS concept evidenced
Parent prints PID and PPID	Process identity and parent relationship.
fork() creates child	Process creation and fork return-path branching.
exec() worker starts with same PID	Program-image replacement distinct from process creation.
Worker waits on OTP pipe	Blocking I/O and interruptible sleep state S.
Correct / incorrect OTP	Normal termination and exit-code reporting.
Decline sends SIGTERM	Signal-based process termination.
waitid(...WNOWAIT) then snapshot	Termination status retained for observation; zombie state Z.
waitpid() returns child PID	Status collection and reaping.

7.6 Validation and Test Results

The repository's validation.txt records the following successful checks:

- Success, PID/PPID relationships, PID continuity across exec, sleeping, zombie state, and reaping.
- Wrong-OTP path with the same process lifecycle checks.
- Decline path with the same process lifecycle checks.
- Interactive UPI, Card, and Net banking flows.
- Interactive decline and input-EOF cleanup.
- Invalid amounts do not create payment children.

The project README also documents web tests for success/failure/decline, input validation, PID/PPID relationships, sleeping, signal status, and cleanup. The report does not claim independent execution of tests during preparation; the test results above are transcribed from the repository's validation record.

CHAPTER - 8 CONCLUSION

8.1 Summary of the Project

The project presents Linux process lifecycle operations through ProcessPay, an educational payment simulation. A parent creates a child with fork(), the child replaces its program image with exec(), pipes coordinate readiness and OTP input, and the parent handles normal completion or signal-based termination. It then observes the zombie state and uses waitpid() to reap the child.

8.2 Objectives Achieved

- A real parent-child process relationship is created and displayed.
- The child PID remains unchanged across exec() in the captured examples.
- The worker blocks while waiting for OTP input through a pipe.
- Correct and incorrect OTPs produce distinct normal exit statuses.
- Decline demonstrates SIGTERM-based termination.
- The parent observes the zombie state before collecting child status.
- Automated integration checks are recorded for the primary lifecycle paths and input handling.

8.3 Results Achieved

The recorded validation reports PASS for the success, wrong-OTP, and decline scenarios; PID/PPID and exec PID continuity checks; sleeping, zombie, and reaping checks; interactive payment modes; decline/EOF cleanup; and invalid amount handling. Captured logs illustrate exit status 0, exit status 1, and signal 15. These results establish the intended educational behaviour as recorded in the repository.

8.4 Skills and Learning Outcomes

- C programming with Linux/POSIX APIs and error handling.
- Understanding process creation versus program-image replacement.
- Using pipes for IPC and coordinating parent/child execution.
- Reading process metadata from /proc.
- Distinguishing blocked input, child waiting, normal exit, signal termination, zombie state, and reaping.
- Writing integration tests and interpreting process exit status.
- Connecting a command-line OS demonstration to an optional local browser interface.

8.5 Limitations and Future Enhancements

The current project is a single-payment educational demonstration. It does not contact banking systems or move funds. OTP retry policies, timeout handling, persistent transaction history, more extensive failure recovery, and cleanup guarantees under forced parent termination are outside the documented scope. Possible future work includes adding an explicit timeout demonstration, extending test coverage for more failure conditions, and improving the UI's accessibility while preserving the distinction between the web layer and actual Linux process operations.

CHAPTER - 9 REFERENCES

9.1 Books

[1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts. Wiley. Refer to the edition prescribed for the course.

[2] W. Richard Stevens and Stephen A. Rago, Advanced Programming in the UNIX Environment. Addison-Wesley.

9.2 Research Papers

No research paper is claimed as a direct source for the implementation. The project is primarily a systems-programming demonstration based on documented Linux/POSIX interfaces. If the course requires research-paper citations, add papers specifically approved by the instructor.

9.3 Linux / GNU Documentation

[3] Linux man-pages project, fork(2), exec(3)/execve(2), pipe(2), read(2), write(2), kill(2), wait(2), waitpid(2), waitid(2), proc(5), and proc_pid_status(5).

[4] GNU Compiler Collection (GCC) documentation.

[5] GNU Make manual.

9.4 Online Resources

[6] Team 08, Linux Process Creation, Execution and Termination System, source code, README, REPORT.md, Makefile, validation.txt, and captured sample outputs. Repository: https://github.com/abhaytripathi-cj7/KLH_CSE_2026
-2027-Team-8-Linux-Process-Creation-Execution-and-Termination-System

[7] The Linux man-pages project: https://man7.org/linux/man-pages/

[8] GNU GCC documentation: https://gcc.gnu.org/onlinedocs/

[9] GNU Make manual: https://www.gnu.org/software/make/manual/